

Exploring DPO's Implicit Reward

Preliminary Results & Challenges — Project Update
Graduate Reinforcement Learning | 14 August 2026

Group Members

#	Name	Student ID	Email
1	«FILL IN»	«FILL IN»	shamima2hossain@gmail.com
2	«FILL IN»	«FILL IN»	«FILL IN»
3	«FILL IN»	«FILL IN»	«FILL IN»

Replace «FILL IN» before submitting.

1 Problem Definition and MDP Specification

Direct Preference Optimization (DPO) reparameterizes the RLHF reward in terms of the policy. Its central claim is that a DPO-trained language model *is* a reward model via the **implicit reward** $\hat{r}(x, y) = \beta \cdot \log[\pi_\theta(y|x) / \pi_{\text{ref}}(y|x)]$. This equivalence is exact only at the optimum and on the training distribution. We ask whether the implicit reward **generalizes as a reward model** as well as an explicitly-trained Bradley–Terry (BT) reward model, under matched data and compute.

1.1 Token-level MDP

Autoregressive generation is a finite-horizon, deterministic MDP $M = (S, A, P, R, \rho_0, \gamma)$:

- **State** $s_t = (x, y_{\leq t})$: the prompt x together with the response tokens generated so far.
- **Action** $a_t = y_t \in V$: emit the next token from the vocabulary V .
- **Transition** $P(s_{t+1}|s_t, a_t)$: deterministic append, $s_{t+1} = (x, y_{\leq t+1})$.
- **Reward** R : sparse and terminal — a scalar $r(x, y)$ delivered at the end-of-sequence token; the KL-regularized RLHF objective adds a per-token shaping term $-\beta \cdot \log[\pi_\theta / \pi_{\text{ref}}]$.
- **Initial state** ρ_0 : a prompt $x \sim D$ with empty response, i.e. $s_0 = (x, \emptyset)$.
- **Terminal states**: a state is terminal once the emitted action is the end-of-sequence token ($a_t = \text{EOS}$) or the length cap H is reached, whichever comes first — so every episode is finite and the return is well defined without discounting.
- **Horizon / discount**: episodic with $H = \max$ new tokens, $\gamma = 1$. The policy is the LM $\pi_\theta(a_t|s_t)$.

Two properties of this MDP matter for the study. The dynamics are *known and deterministic* (appending a token), so there is no transition-model uncertainty to estimate — all difficulty sits in the reward. And the reward is *sparse and terminal*: no learning signal arrives until the response is complete, which is precisely why preference-based methods replace an explicit per-token reward with a comparison between whole trajectories.

1.2 Contextual-bandit abstraction (DPO's view)

DPO collapses the horizon: the context is the prompt $x \sim D$, the action is the *entire* response $y \sim \pi(\cdot|x)$, and the reward is the terminal scalar $r(x, y)$. The KL-regularized objective is

$$\max_{\pi} E_{x \sim D, y \sim \pi(\cdot|x)} [r(x, y)] - \beta \cdot D_{KL}(\pi(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)).$$

This KL-constrained problem has the closed-form optimum

$$\pi^*(y|x) = (1/Z(x)) \cdot \pi_{\text{ref}}(y|x) \cdot \exp(r(x,y)/\beta),$$

which, solved for the reward, gives the reparameterization $r(x,y) = \beta \cdot \log[\pi^*(y|x)/\pi_{\text{ref}}(y|x)] + \beta \cdot \log Z(x)$. The partition term $\beta \cdot \log Z(x)$ depends on x only, so it cancels in any same-prompt pairwise comparison.

1.3 Preference model, DPO loss, and the two scorers

Preferences follow the Bradley–Terry model $P(y_w > y_l | x) = \sigma(r(x,y_w) - r(x,y_l))$, where $y_w > y_l$ denotes that y_w is preferred to y_l . Substituting the reparameterized reward ($Z(x)$ cancels) yields the **DPO loss**

$$L_{\text{DPO}} = -E_{(x,y_w,y_l)} \log \sigma(\beta \log[\pi_\theta(y_w|x)/\pi_{\text{ref}}(y_w|x)] - \beta \log[\pi_\theta(y_l|x)/\pi_{\text{ref}}(y_l|x)]).$$

Implicit reward (object of study). $\hat{r}_\theta(x,y) = \beta \log[\pi_\theta(y|x)/\pi_{\text{ref}}(y|x)] = \beta \sum_t (\log \pi_\theta(y_t|x,y_{<t}) - \log \pi_{\text{ref}}(y_t|x,y_{<t}))$.

Explicit reward (baseline). A scalar head $r_\phi(x,y)$ trained with the BT loss $L_{\text{RM}} = -E \log \sigma(r_\phi(x,y_w) - r_\phi(x,y_l))$ on the identical pairs, from the identical SFT initialization.

Evaluation as a preference classifier. For a held-out pair, predict $y_w > y_l$ iff $r(x,y_w) > r(x,y_l)$; the primary metric is **pairwise accuracy**. Note ranking is invariant to β (a positive scalar), so β affects calibration/ECE, not accuracy.

Research question & hypotheses. Under matched data/compute, how does held-out preference accuracy of $\hat{r}$ compare to r_ϕ , ID and under shift? **H1:** explicit > implicit on held-out ID pairs. **H2:** the gap widens under distribution shift. A null result (implicit $\approx$ explicit) would empirically substantiate DPO's claim at small scale.

2 Benchmark Datasets

Following the instructor's feedback to use benchmark datasets, all data are established public benchmarks:

Dataset	Role in this study
UltraFeedback-binarized (HuggingFaceH4/ultrafeedback_binarized)	Training preference pairs and the in-distribution (ID) held-out test set.
RewardBench (allenai/reward-bench)	The standard reward-model evaluation benchmark; its Chat / Chat-Hard / Safety / Reasoning subsets provide controlled distribution shift (OOD).
Anthropic HH-RLHF (Anthropic/hh-rlhf)	A classic human-preference benchmark used as a cross-dataset OOD set.

3 Initial Baselines: Implementations and Results

3.1 Reference baselines (implemented and run)

These non-neural baselines are the reference floors the neural reward models must beat. They require no GPU and are already evaluated on every benchmark set:

- **Random** — chance (0.500).
- **Pick-longer / Pick-shorter** — predict the longer (resp. shorter) response wins; probes the length confound flagged in the proposal.
- **TF-IDF + Logistic Regression** — a learned lexical scorer: TF-IDF over (prompt, response), trained on UltraFeedback to score a response chosen/rejected, higher-scoring side wins.

Test set	n	Random	Pick-longer	Pick-shorter	TF-IDF+LR
UltraFeedback (ID)	1997	0.500	0.568	0.432	0.618
HH-RLHF harmless (OOD)	2312	0.500	0.445	0.555	0.453

RewardBench: Chat (OOD)	358	0.500	0.804	0.196	0.754
RewardBench: Chat-Hard (OOD)	456	0.500	0.294	0.706	0.382
RewardBench: Safety (OOD)	740	0.500	0.420	0.580	0.450
RewardBench: Reasoning (OOD)	1431	0.500	0.397	0.603	0.282
RewardBench: ALL (OOD)	2985	0.500	0.436	0.564	0.395

Pairwise accuracy. TF-IDF+LR trained on 19,980 UltraFeedback pairs (seed 0). Ties count as 0.5.

3.2 Key preliminary findings

- **Length bias is large and dataset-dependent.** Pick-longer reaches 0.804 on RewardBench-Chat but only 0.294 on the adversarial Chat-Hard (which deliberately inverts length). This validates the proposal's length-controlled metric as essential — a scorer can look strong purely by proxying length.
- **A lexical learner transfers partially.** TF-IDF+LR beats chance in-distribution (0.618) and on easy Chat (0.754), but collapses to near/below chance on Chat-Hard (0.382) and Reasoning (0.282) — quantifying how much of the signal is shallow lexical overlap and setting a floor for the neural models.
- The ID→OOD accuracy drop already visible here previews exactly the H2 comparison we will make between the implicit and explicit rewards.

3.3 Neural pipeline: now running on GPU

Since the last update the pipeline has moved off the drawing board and onto hardware: an AWS SageMaker instance with NVIDIA A10G GPUs (24 GB, Ampere → native bf16), replacing the Colab plan. Qwen2.5-0.5B + LoRA via HuggingFace TRL, as designed. The stage status below is read directly from each checkpoint's *budget.json*, which the trainers now write automatically so the matched-compute claim is auditable rather than asserted.

Stage	Status	Steps	Epochs	Eff. batch	Wall-clock
SFT → π_{ref} (32k)	complete	1998	1.00	16	1h 13m
DPO $\beta=0.1$ (8k)	complete	500	1.00	16	42m 21s
Bradley–Terry RM (8k)	complete	470	1.00	16	23m 19s

π_{ref} is trained: one full epoch over 32k UltraFeedback pairs, 1998 optimizer steps at effective batch 16, LoRA $r=16$. DPO and the RM both initialize from it, so neither scorer gets a head start.

Evaluation reports pairwise accuracy (primary), ECE, length-controlled accuracy, Spearman vs gold, and a McNemar paired test between the implicit and explicit scorers (*src/metrics.py*, *src/eval.py*). The harness has been validated end-to-end on a reduced run. All three training stages are complete and the full evaluation over the ID and OOD benchmark sets is currently running; H1/H2 numbers follow on its completion.

Known caveat on the compute match. The RM completed 470 optimizer steps (7,520 pairs) against DPO's 500 (8,000) at the same nominal 8k budget and identical effective batch. The cause is a difference in how the two TRL trainers handle over-length examples: RewardTrainer *drops* pairs exceeding `max_length` while DPOTrainer *truncates* them. The two conditions therefore do not see byte-identical data, which weakens the 'same pairs' half of our matched-compute definition. We are pre-filtering both conditions to a common length-eligible subset and will re-run before reporting headline H1/H2 numbers; the figures below should be read with this in mind.

3.4 Two correctness bugs found before trusting any number

Bringing the pipeline up on GPU surfaced two defects that would not have crashed — they would have silently produced plausible but meaningless H1/H2 results. Both stemmed from treating the SFT LoRA *adapter* as the shared initialization:

- **The DPO reference was the wrong model.** With a PEFT policy and `ref_model=None`, TRL derives π_{ref} by *disabling the adapter* — which returns the raw base model, not the SFT checkpoint. Training therefore optimized $\beta \cdot (\log \pi_{\theta} - \log \pi_{\text{base}})$ while evaluation scored against base+SFT: two different quantities, neither the π_{ref} the design specifies.
- **The reward model's scalar head never trained.** Loading a CAUSAL_LM adapter onto a sequence-classification model leaves the freshly initialized *score* head frozen at random values (the adapter carries no `modules_to_save`). Measured directly: `score head trainable = False`. The explicit RM — the entire H1 baseline — would have been a LoRA trained underneath a random frozen projection.

Both are fixed by materializing π_{ref} once as a merged checkpoint (`src/merge_sft.py`). DPO then trains a fresh zero-initialized LoRA on it, so $\pi_{\theta} = \pi_{\text{ref}}$ at step 0 and the implicit reward starts at exactly 0 — confirmed empirically by the DPO loss beginning at $\ln 2 \approx 0.693$. The RM adds a fresh SEQ_CLS LoRA, after which the head is trainable and saved. Evaluation loads the same merged checkpoint, so training and evaluation agree by construction.

The implicit-reward extraction is additionally cross-checked against DPOTrainer's own reward computation (`src/verify_implicit.py`): tokenization matches exactly, the reward is exactly linear in β , and the residual disagreement is an order of magnitude smaller than the shift a prompt-masking bug would produce. This guards the quantity the entire study measures.

4 Challenges

- **Auditable budget-matching.** The comparison is only meaningful if DPO and the RM see identical data/compute. We fix a shared SFT initialization and match epochs, LoRA config, batch, and LR, and log tokens-seen / wall-clock — but defining 'equal compute' across two different losses (DPO's extra reference forward pass) remains a judgment call we must defend.
- **Correct implicit-reward extraction.** The β -scaled, prompt-masked, length-summed log-ratio must exactly match TRL's internal convention; a subtle masking or scaling error would silently bias every downstream number. *Addressed* (Section 3.4): cross-checked against DPOTrainer's own reward computation, with explicit controls for the masking and β -scaling failure modes.
- **Silent failures beat loud ones.** The two defects in Section 3.4 both ran without error and produced numbers that looked reasonable. The lesson we are carrying forward is to verify each component against an independent implementation before trusting any result it feeds.
- **Small-model noise.** At 0.5B, reward accuracy can sit close to chance on hard subsets (the TF-IDF collapse on Reasoning previews this), so headline claims need ≥ 3 seeds with confidence intervals; we may escalate to 1.5B if the signal is too noisy.
- **Length as a confound.** Both scorers can proxy length (Section 3.2); isolating true preference signal requires the length-controlled metric and reporting the length-only baseline everywhere.
- **Gold-judge / Spearman set on limited compute.** A larger instruction-tuned judge for scalar gold ratings must be run as a separate quantized/offline pass (or via API) to fit a single consumer GPU alongside training.
- **Toolchain drift — encountered, not hypothetical.** Standing the environment up required upper-bounding *transformers* (TRL 0.15 imports a symbol removed in transformers 5.x, so an unbounded pin broke the import outright) and handling an SFTConfig argument that was renamed across TRL releases. Versions are now pinned on both sides in `requirements.txt`.
- **Hardware-dependent silent misconfiguration.** On a multi-GPU host, HuggingFace Trainer auto-wraps the model in DataParallel, which quadrupled the effective batch size — breaking the matched-compute definition without any warning — and ran 6× slower. Runs are now pinned to one GPU so the executed configuration matches the documented one.

5 Next Steps

- **Immediate.** Finish DPO ($\beta=0.1$) and the BT reward model at the 8k budget from the shared π_{ref} , then run the evaluation harness to produce the first H1/H2 numbers (implicit vs explicit, ID + OOD) against the reference floors in Section 3.1.
- Re-run the implicit-reward cross-check on the trained checkpoints, where the rewards are far from zero and the comparison is most informative.
- Widen to the ablations: $\beta \in \{0.05, 0.1, 0.3, 0.5\}$, budget $\in \{2k, 8k, 32k\}$, and the training-progress checkpoint curve (overoptimization).
- Add the gold-judge OOD set and Spearman correlation; finalize with ≥ 3 seeds and significance tests.

Code, benchmark data pipeline, and reproducible baseline results: github.com/silvererudite/dpo-implicit-reward